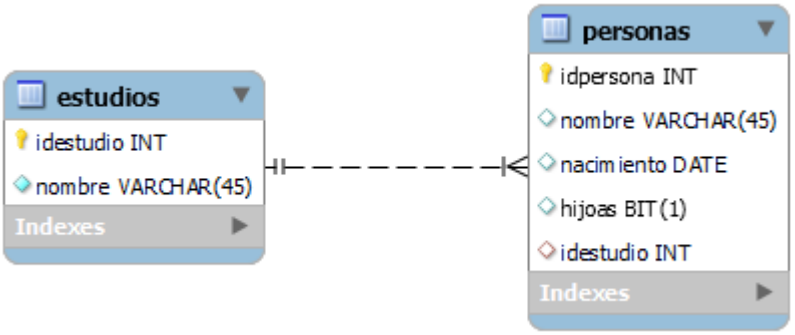
Ejercitación

Realiza alta, baja, modificación y consulta. Las pantallas, el modelo y el guión de la base están a continuación.

Base de datos




```
-- MySQL Workbench Forward Engineering
-- -----------------------------------------------------
-- Schema recuperacion
-- -----------------------------------------------------
DROP SCHEMA IF EXISTS `tpFinalPOO` ;

-- -----------------------------------------------------
-- Schema tpFinalPOO
-- -----------------------------------------------------
CREATE SCHEMA IF NOT EXISTS `tpFinalPOO` DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci ;
USE `tpFinalPOO` ;

-- -----------------------------------------------------
-- Table `tpFinalPOO`.`estudios`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `tpFinalPOO`.`estudios` ;

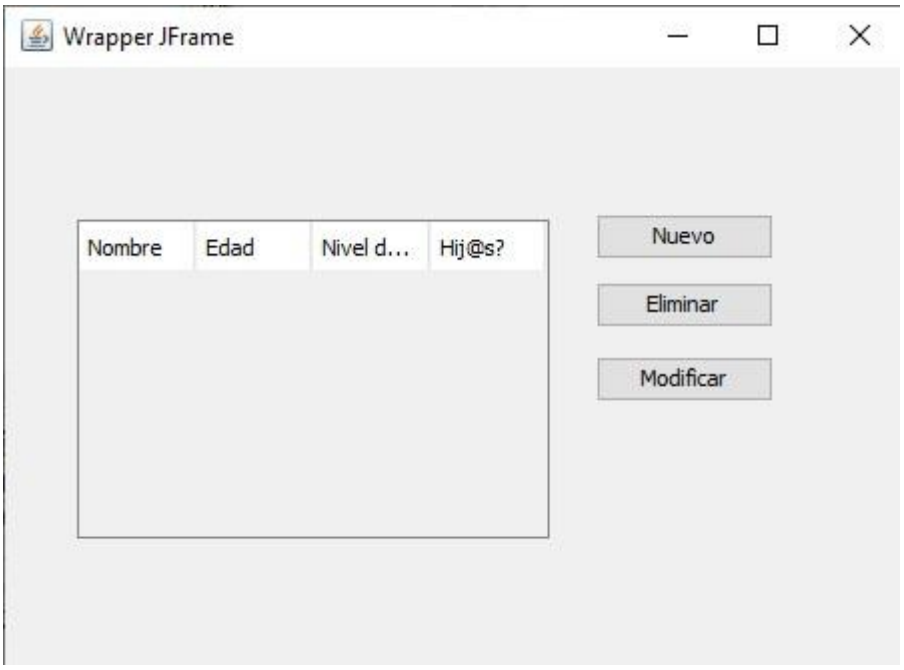
CREATE TABLE IF NOT EXISTS `tpFinalPOO`.`estudios` (
  `idestudio` INT NOT NULL,
  `nombre` VARCHAR(45) NOT NULL,
  PRIMARY KEY (`idestudio`));

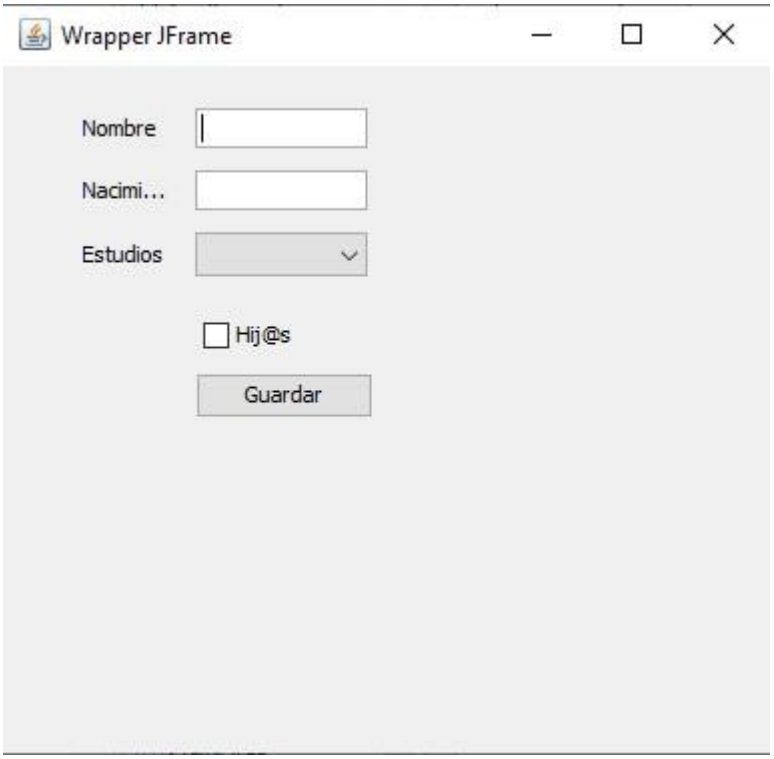

-- -----------------------------------------------------
-- Table `tpFinalPOO`.`personas`
-- -----------------------------------------------------
DROP TABLE IF EXISTS `recuperacion`.`personas` ;
```

```
CREATE TABLE IF NOT EXISTS `tpFinalPOO`.`personas` (
  `idpersona` INT NOT NULL AUTO_INCREMENT,
  `nombre` VARCHAR(45) NULL DEFAULT NULL,
  `nacimiento` DATE NULL DEFAULT NULL,
  `hijoas` BIT(1) NULL DEFAULT NULL,
  `idestudio` INT NULL DEFAULT NULL,
  PRIMARY KEY (`idpersona`),
  INDEX `fk_estudio_idx` (`idestudio` ASC),
  CONSTRAINT `fk_estudio`
        FOREIGN KEY (`idestudio`)
        REFERENCES `tpFinalPOO`.`estudios` (`idestudio`)
        ON DELETE CASCADE);

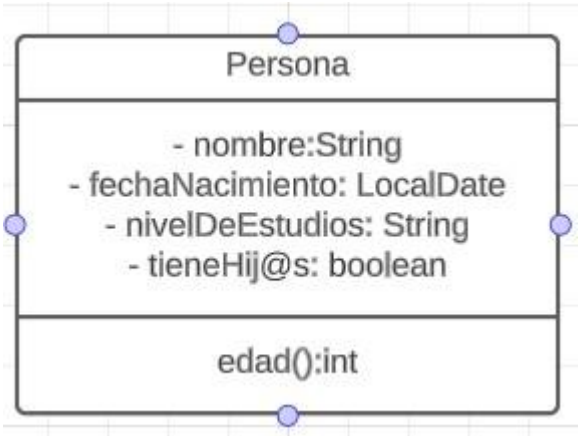
INSERT INTO `tpFinalPOO`.`estudios` (`idestudio`, `nombre`) VALUES (1, 'Secundario
incompleto');
INSERT INTO `tpFinalPOO`.`estudios` (`idestudio`, `nombre`) VALUES (2, 'Secundario
completo');
INSERT INTO `tpFinalPOO`.`estudios` (`idestudio`, `nombre`) VALUES (3, 'Terciario no
universitario');
INSERT INTO `tpFinalPOO`.`estudios` (`idestudio`, `nombre`) VALUES (4, 'Universitario');
INSERT INTO `tpFinalPOO`.`estudios` (`idestudio`, `nombre`) VALUES (5, 'Universitario
incompleto');
```

Pantallas





Modelo



Requerimientos no funcionales

1. Toda la lógica de negocio debe estar en la aplicación (las consultas SQL no resuelven problemas).
2. Las consultas SQL son simples. No se permite el uso de subconsultas.
3. La identidad de los objetos está dada por uno o más atributos pero no por “id” de la tabla. Los objetos no pueden tener el atributo “id”.
4. La interfaz de escritorio utiliza Swing. Existirá un solo objeto de tipo JFrame en toda la aplicación.

5. La pantalla para el alta y la modificación es la misma (resolver el alta y la modificación con la misma clase).
6. La aplicación utiliza objetos de la clase JTable para listar elementos, JComboBox para desplegar múltiples elementos (por ej. categorías) y JList para seleccionar múltiples elementos.
7. El sistema utiliza las clases que están en el paquete java.time.* para manejar fechas.
8. La aplicación usa JDBC para el acceso a la base de datos relacional (no usa frameworks).
9. Se deberán programar excepciones para los casos extraordinarios (Por ejemplo, no puedo modificar o eliminar algo sin haber seleccionado un registro anteriormente). Utilizar ventanas emergentes, no retornos por consola.

Sobre el desarrollo y la corrección

El ejercicio se desarrolla durante la clase pudiendo ocupar más de un día.
No se puede adelantar el ejercicio “en casa”. Para aprobar debe estar codificado según la especificación, estar completo, implementar los patrones de diseño DAO y MVC y funcionar correctamente.

El orden de desarrollo será el siguiente:

1. Alta sin nivel de estudios
2. Consulta sin nivel de estudios.
3. Eliminación.
4. Incorporar el nivel de estudios al alta y la consulta.
5. Modificación.

Estos ítems son puntos de control: una vez terminado el punto, será evaluado y recién ahí, si está correcto, podrá pasar al siguiente ítem.